ComputeModule — Complete SystemC Implementation Guide

This guide explains everything needed to implement `dependencies_received()` in **compute_module.cpp** — the only function that needs to be filled in. It explains why new memory arrays are needed, where they come from, and how each of the 10 instruction types is handled.

1. Why New Memory Arrays Are Needed

In `vta.cc`, the top-level `vta()` function declares three SRAM buffers and passes them as parameters to all three stages:

From vta.cc — vta() function:

```
bus_T inp_mem[VTA_INP_BUFF_DEPTH][INP_MAT_AXI_RATIO];   // shared: Load writes, Compute reads
bus_T wgt_mem[VTA_WGT_BUFF_DEPTH][WGT_MAT_AXI_RATIO];   // shared: Load writes, Compute reads
bus_T out_mem[VTA_ACC_BUFF_DEPTH][OUT_MAT_AXI_RATIO];    // shared: Compute writes, Store reads

load(inputs, weights, load_queue, ..., inp_mem, wgt_mem);          // Load fills inp and wgt
compute(done, uops, biases, gemm_queue, ..., inp_mem, wgt_mem, out_mem);  // Compute uses all
store(outputs, store_queue, ..., out_mem);                          // Store reads out
```

From vta.cc — compute() function (private SRAMs):

```
static uop_T uop_mem[VTA_UOP_BUFF_DEPTH];                    // PRIVATE to Compute — micro-op table
static bus_T acc_mem[VTA_ACC_BUFF_DEPTH][ACC_MAT_AXI_RATIO]; // PRIVATE to Compute — accumulator
```

These 5 arrays ARE the VTA hardware's on-chip SRAM buffers. Without them, GEMM has no data to multiply and ALU has no accumulator to operate on. They must exist in the SystemC version.

In the SystemC simulator the `bus_T = ap_uint<512>` wide AXI bus format is unnecessary — we have no real bus. So we use simple, unpacked element arrays instead. This also means we don't need the `read_tensor` / `write_tensor` functions from vta.cc:

vta.cc (HLS — wide bus format)	SystemC (simplified — element arrays)
`bus_T uop_mem[VTA_UOP_BUFF_DEPTH]` where bus_T = ap_uint<32> (uop is 32-bit)	`uint32_t uop_mem[UOP_BUFF_DEPTH]`
`bus_T acc_mem[VTA_ACC_BUFF_DEPTH][ACC_MAT_AXI_RATIO]` + read_tensor/write_tensor unpacking	`int32_t acc_mem[ACC_BUFF_DEPTH][VTA_BLOCK_OUT]` (already unpacked: 1 tile = 16 int32 values)
`bus_T inp_mem[VTA_INP_BUFF_DEPTH][INP_MAT_AXI_RATIO]`	`int8_t inp_mem[INP_BUFF_DEPTH][VTA_BLOCK_IN]` (already unpacked: 1 tile = 16 int8 values)

+ read_tensor unpacking	
`bus_T wgt_mem[VTA_WGT_BUFF_DEPTH][WGT_MAT_AXI_RATIO]` + read_tensor unpacking	`int8_t wgt_mem[WGT_BUFF_DEPTH][VTA_BLOCK_OUT * VTA_BLOCK_IN]` (1 tile = 16×16 int8 values, flattened)
`bus_T out_mem[VTA_ACC_BUFF_DEPTH][OUT_MAT_AXI_RATIO]` + write_tensor packing	`int8_t out_mem[ACC_BUFF_DEPTH][VTA_BLOCK_OUT]` (already unpacked: 1 tile = 16 int8 values)

2. All 10 Instruction Types That Compute Handles

Reading the actual CSV (resnet_101.csv), these are the exact instruction names that go to the Compute module:

Name in CSV	What it does in vta.cc compute()	In SystemC
`LOAD UOP`	Loads micro-op table from DRAM into `uop_mem`. `memcpy(&uop_mem[sram_idx], &uops[dram_idx], x_size * sizeof(uop_T))`	Extract sram_idx and x_size from instruction. No real DRAM in simulator — note DRAM access needed.
`LOAD ACC`	Loads bias/accumulator data from DRAM into `acc_mem` with padding. `load_pad_2d(biases, acc_mem, sram_idx, dram_idx, y_size, x_size, stride, ...)`	Extract parameters from instruction. No real DRAM in simulator — acc_mem stays zero (zero bias).
`GEMM`	Matrix multiply: reads `uop_mem`, `inp_mem`, `wgt_mem`, reads/writes `acc_mem`, writes `out_mem`.	Full loop implementation. See Section 4.
`ALU - add`	Element-wise add: dst_tile += src_tile. use_imm = false.	ALU loop implementation using instruction name to detect opcode. See Section 5.
`ALU - add imm`	Element-wise add with immediate: dst_tile += imm. use_imm = true.	
`ALU - max imm`	Element-wise max(dst, imm). Usually imm=0 → ReLU. use_imm = true.	
`ALU - min imm`	Element-wise min(dst, imm). Clamping. use_imm = true.	
`ALU - shr`	Arithmetic right shift: dst >>= src. Fixed-point re-quantization.	
`NOP-COMPUTE-STAGE`	No operation. Just a synchronisation point.	No computation. Only timing: `finish.notify(latency())`

`FINISH`	Sets done flag, signals end of layer.	Already handled by `finalize_instruction()` — prints "FINISH LAYER" message.

3. Constants and Memory Declarations — Top of compute_module.cpp

These go immediately after the `#include` lines, before any function body. They do NOT go in any header file.

```
#include "compute_module.h"
#include <cstring>   // for std::memcpy
#include <algorithm> // for std::max, std::min

// ─── VTA hardware block sizes (from VTA hw_spec.h — not present in simulator) ─
// VTA_BLOCK_IN/OUT: number of elements per tile edge (input and output channels)
// These match the VTA hardware configuration used to compile the ResNet models.
constexpr int VTA_BATCH     = 1;   // input batch dimension per tile (standard VTA = 1)
constexpr int VTA_BLOCK_IN  = 16;  // input channel dimension per tile
constexpr int VTA_BLOCK_OUT = 16;  // output channel dimension per tile

// ─── Buffer depths derived from vta_config.h bit-width constants ──────────────
// UOP_DST_WIDTH = 11 → 2^11 = 2048 possible DST indices → acc/inp buffer depth
// UOP_WGT_WIDTH = 10 → 2^10 = 1024 possible WGT indices → wgt buffer depth
constexpr int UOP_BUFF_DEPTH = (1 << vta_config::UOP_DST_WIDTH);  // 2048 UOP entries
constexpr int ACC_BUFF_DEPTH = (1 << vta_config::UOP_DST_WIDTH);  // 2048 acc tiles
constexpr int INP_BUFF_DEPTH = (1 << vta_config::UOP_SRC_WIDTH);  // 2048 input tiles
constexpr int WGT_BUFF_DEPTH = (1 << vta_config::UOP_WGT_WIDTH);  // 1024 weight tiles

// ─── Bitmasks for extracting indices from a 32-bit micro-op word ──────────────
// UOP bit layout: [10:0]=dst(11b) [21:11]=src(11b) [31:22]=wgt(10b)
constexpr uint32_t DST_MASK = (1u << vta_config::UOP_DST_WIDTH) - 1u;  // 0x7FF
constexpr uint32_t SRC_MASK = (1u << vta_config::UOP_SRC_WIDTH) - 1u;  // 0x7FF
constexpr uint32_t WGT_MASK = (1u << vta_config::UOP_WGT_WIDTH) - 1u;  // 0x3FF

// ─── SRAM buffers (mirrors of vta.cc's hardware on-chip memory) ───────────────
// uop_mem, acc_mem: PRIVATE to Compute (vta.cc declares them static inside compute())
// inp_mem, wgt_mem, out_mem: SHARED — Load fills inp/wgt, Store reads out
//   (in vta.cc these are declared in vta() and passed as parameters to all stages)

// Private to Compute:
static uint32_t uop_mem[UOP_BUFF_DEPTH];                              // micro-op SRAM
static int32_t  acc_mem[ACC_BUFF_DEPTH][VTA_BLOCK_OUT];               // accumulator SRAM (int32)

// Shared with Load (writes) and Store (reads):
// Declared without 'static' so they are global and accessible via 'extern' from other .cpp files
int8_t inp_mem[INP_BUFF_DEPTH][VTA_BLOCK_IN];                         // input SRAM (int8)
int8_t wgt_mem[WGT_BUFF_DEPTH][VTA_BLOCK_OUT * VTA_BLOCK_IN];         // weight SRAM (int8, flattened)
int8_t out_mem[ACC_BUFF_DEPTH][VTA_BLOCK_OUT];                        // output SRAM (int8)
```

Why static vs non-static?

`static` at file scope = private to this .cpp file (like C's file-local variables). Use it for `uop_mem` and `acc_mem` which only Compute ever touches, matching vta.cc's `static` declaration inside `compute()`.

`inp_mem`, `wgt_mem`, `out_mem` must be accessible from `load_module.cpp` and `store_module.cpp`. Without `static`, they are global symbols. The other .cpp files can access them with:

> `extern int8_t inp_mem[][16];` (placed at the top of load_module.cpp / store_module.cpp — no .h change needed)

4. LOAD UOP — Loading the Micro-Op Table

vta.cc (exact code inside compute() LOAD branch):

```
  // When memory_type == VTA_MEM_ID_UOP:
  memcpy(&uop_mem[sram_idx],
         (const uop_T*) &uops[dram_idx],
         insn.mem.x_size * sizeof(uop_T));
  // Where: sram_idx = insn.mem.sram_base  (SRAM destination offset)
  //        dram_idx = insn.mem.dram_base  (DRAM source address)
  //        x_size = number of UOPs to copy
```

```
// ── LOAD UOP: fill uop_mem from DRAM ──────────────────────────────
if (name == "LOAD UOP") {
    // sram_base: where in uop_mem to write (hex string → integer)
    int sram_base = (int)std::stoul(current->get_sram(), nullptr, 16);
    int x_size    = current->get_x_size();   // number of UOP words to load

    // In vta.cc: memcpy(&uop_mem[sram_base], &uops[dram_base], x_size * sizeof(uint32_t))
    // In this simulator there is no DRAM pointer — uop data is not available at runtime.
    // uop_mem remains zero-initialized. GEMM/ALU will use zero UOPs (indices all 0).
    // A full functional simulator would need the UOP binary loaded from a file here.
    (void)sram_base; (void)x_size;
}
```

What is a micro-op?

A micro-op (UOP) is a 32-bit word that tells GEMM or ALU which SRAM tiles to use for one computation step: [bits 10:0] = destination tile index, [bits 21:11] = source/input tile index, [bits 31:22] = weight tile index. The UOP table is loaded once before GEMM/ALU instructions run.

5. LOAD ACC — Loading Bias Data

vta.cc (exact code inside compute() LOAD branch):

```
  // When memory_type == VTA_MEM_ID_ACC:
  load_pad_2d<bus_T, ACC_MAT_AXI_RATIO, VTA_ACC_ELEM_BYTES>(
      biases, acc_mem, sram_idx, dram_idx,
      insn.mem.y_size, insn.mem.x_size, insn.mem.x_stride,
      insn.mem.x_pad_0, insn.mem.x_pad_1, y_offset_0, y_offset_1);
  // Loads bias tensors from DRAM into acc_mem with optional zero-padding.
  // acc_mem is pre-loaded with bias before GEMM runs (insn.reset_reg = false keeps the bias).
```

```
// ── LOAD ACC: pre-load accumulator (bias) from DRAM ──────────────────
else if (name == "LOAD ACC") {
    int sram_base = (int)std::stoul(current->get_sram(), nullptr, 16);
    int y_size    = current->get_y_size();
    int x_size    = current->get_x_size();
```

```
    int x_stride  = current->get_stride();

    // In vta.cc: load_pad_2d copies bias data into acc_mem[sram_base .. sram_base+tiles]
    // In this simulator: no DRAM bias pointer — acc_mem initialized to 0 (zero bias).
    // A full functional simulator would load the bias tensor from a file here.
    (void)sram_base; (void)y_size; (void)x_size; (void)x_stride;
}
```

6. GEMM — Matrix Multiply Implementation

What the HLS gemm() function does (step by step):

1. Outer loop: iterates `iter_out` times, tracking tile offsets for dst/src/wgt
2. Inner loop: iterates `iter_in` times, adding inner step factors
3. Micro-op loop: for each UOP in [uop_bgn, uop_end), decode tile indices from UOP word
4. For each output channel (oc=0..15): read accumulator, dot-product over 16 input channels, update accumulator
5. Write back to acc_mem (with optional reset) and to out_mem (truncated to int8)

Field mapping: vta.cc gemm() → Instruction* getters

vta.cc field	SystemC getter
`insn.iter_out`	`current->get_outer_loop_iter()`
`insn.iter_in`	`current->get_inner_loop_iter()`
`insn.uop_bgn`	`current->get_range_0()`
`insn.uop_end`	`current->get_range_1()`
`insn.dst_factor_out`	`current->get_gemm_outer_loop_acc()`
`insn.src_factor_out`	`current->get_gemm_outer_loop_inp()`
`insn.wgt_factor_out`	`current->get_gemm_outer_loop_wgt()`
`insn.dst_factor_in`	`current->get_gemm_inner_loop_acc()`
`insn.src_factor_in`	`current->get_gemm_inner_loop_inp()`
`insn.wgt_factor_in`	`current->get_gemm_inner_loop_wgt()`
`insn.reset_reg`	`current->get_reset_out()`
`uop.range(VTA_UOP_GEM_0_1, VTA_UOP_GEM_0_0)` — dst bits	`(uop >> 0) & DST_MASK`
`uop.range(VTA_UOP_GEM_1_1, VTA_UOP_GEM_1_0)` — src bits	`(uop >> UOP_DST_WIDTH) & SRC_MASK`
`uop.range(VTA_UOP_GEM_2_1, VTA_UOP_GEM_2_0)` — wgt bits	`(uop >> (UOP_DST_WIDTH+UOP_SRC_WIDTH)) & WGT_MASK`
`read_tensor(..., wgt_mem, w_tensor)` → `w_tensor[oc][ic]`	`wgt_mem[wgt_idx][oc * VTA_BLOCK_IN + ic]`
`read_tensor(..., inp_mem, i_tensor)` → `i_tensor[b][ic]`	`inp_mem[src_idx][ic]`
`read_tensor(..., acc_mem, a_tensor)` → `a_tensor[b][oc]`	`acc_mem[dst_idx][oc]`

`accum.range(VTA_OUT_WIDTH-1, 0)` — keep 8 bits	`accum & 0xFF`

SystemC GEMM code:

```
// ── GEMM ─────────────────────────────────────────────────────────────────
// Mirrors: EXE_OUT_LOOP > EXE_IN_LOOP > READ_GEMM_UOP > inner MAC from vta.cc gemm()
else if (name == "GEMM") {
    int dst_offset_out = 0;   // vta.cc: acc_idx_T dst_offset_out = 0
    int src_offset_out = 0;   // vta.cc: inp_idx_T src_offset_out = 0
    int wgt_offset_out = 0;   // vta.cc: wgt_idx_T wgt_offset_out = 0

    // EXE_OUT_LOOP
    for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
        int dst_offset_in = dst_offset_out;
        int src_offset_in = src_offset_out;
        int wgt_offset_in = wgt_offset_out;

        // EXE_IN_LOOP
        for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

            // READ_GEMM_UOP — iterate over micro-op table slice [range_0, range_1)
            for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                uint32_t uop = uop_mem[upc];   // vta.cc: uop_T uop = uop_mem[upc]

                // Decode tile indices from UOP word (replaces HLS uop.range())
                int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
                int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;
                int wgt_idx = (int)((uop >> (vta_config::UOP_DST_WIDTH + vta_config::UOP_SRC_WIDTH)) & WGT_MA

                // Inner MAC loop (VTA_BATCH=1, so b=0 is the only batch)
                for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {
                    int32_t accum = acc_mem[dst_idx][oc];  // vta.cc: a_tensor[b][oc]
                    int32_t tmp   = 0;                     // vta.cc: sum_T tmp = 0

                    for (int ic = 0; ic < VTA_BLOCK_IN; ic++) {
                        // vta.cc: wgt_T w_elem = w_tensor[oc][ic]
                        int8_t w = wgt_mem[wgt_idx][oc * VTA_BLOCK_IN + ic];
                        // vta.cc: inp_T i_elem = i_tensor[b][ic]
                        int8_t x = inp_mem[src_idx][ic];
                        tmp += (int32_t)x * (int32_t)w;   // vta.cc: mul_T prod; tmp += prod
                    }

                    accum += tmp;

                    // vta.cc: a_tensor[b][oc] = insn.reset_reg ? (acc_T) 0 : accum
                    acc_mem[dst_idx][oc] = current->get_reset_out() ? 0 : accum;

                    // vta.cc: o_tensor[b][oc] = (out_T) accum.range(VTA_OUT_WIDTH-1, 0)
                    // .range(7, 0) = keep lowest 8 bits = & 0xFF
                    out_mem[dst_idx][oc] = (int8_t)(accum & 0xFF);
                }
            }

            // Update inner offsets — vta.cc: dst_offset_in += insn.dst_factor_in; etc.
            dst_offset_in += current->get_gemm_inner_loop_acc();
            src_offset_in += current->get_gemm_inner_loop_inp();
            wgt_offset_in += current->get_gemm_inner_loop_wgt();
```

```
        }

        // Update outer offsets — vta.cc: dst_offset_out += insn.dst_factor_out; etc.
        dst_offset_out += current->get_gemm_outer_loop_acc();
        src_offset_out += current->get_gemm_outer_loop_inp();
        wgt_offset_out += current->get_gemm_outer_loop_wgt();
    }
}
```

7. ALU — Element-Wise Accumulator Operations

What the HLS alu() function does:

Same outer/inner/UOP loop structure as GEMM, but operates on two accumulator tiles (no weight memory). The operation applied depends on `insn.alu_opcode`. In SystemC we detect the opcode from the instruction name string since there is no `get_alu_opcode()` getter.

Key difference from GEMM:

- **No weight memory** — ALU operates on two acc_mem tiles (dst and src)
- **use_imm** — if true, src_1 is an immediate value (not from acc_mem). Since the Instruction class has no `get_imm()` getter, we detect "imm" in the name and assume the immediate = 0. This correctly covers the most common case: `ALU - max imm 0` = ReLU (max(x, 0)).

Field mapping: vta.cc alu() → Instruction* getters

vta.cc field	SystemC getter
`insn.iter_out`	`current->get_outer_loop_iter()`
`insn.iter_in`	`current->get_inner_loop_iter()`
`insn.uop_bgn`	`current->get_range_0()`
`insn.uop_end`	`current->get_range_1()`
`insn.dst_factor_out`	`current->get_alu_outer_loop_dst()`
`insn.src_factor_out`	`current->get_alu_outer_loop_src()`
`insn.dst_factor_in`	`current->get_alu_inner_loop_dst()`
`insn.src_factor_in`	`current->get_alu_inner_loop_src()`
`insn.use_imm`	name contains "imm" → true
`insn.imm`	0 (no getter available; covers ReLU correctly)
`insn.alu_opcode == VTA_ALU_OPCODE_MAX`	name contains "max"
`insn.alu_opcode == VTA_ALU_OPCODE_MIN`	name contains "min"
`insn.alu_opcode == VTA_ALU_OPCODE_ADD`	name contains "add"
`insn.alu_opcode == VTA_ALU_OPCODE_SHR`	name contains "shr"
`dst_tensor[i][b]` — first operand	`acc_mem[dst_idx][oc]`
`src_tensor[i][b]` — second operand (if !use_imm)	`acc_mem[src_idx][oc]`

SystemC ALU code:

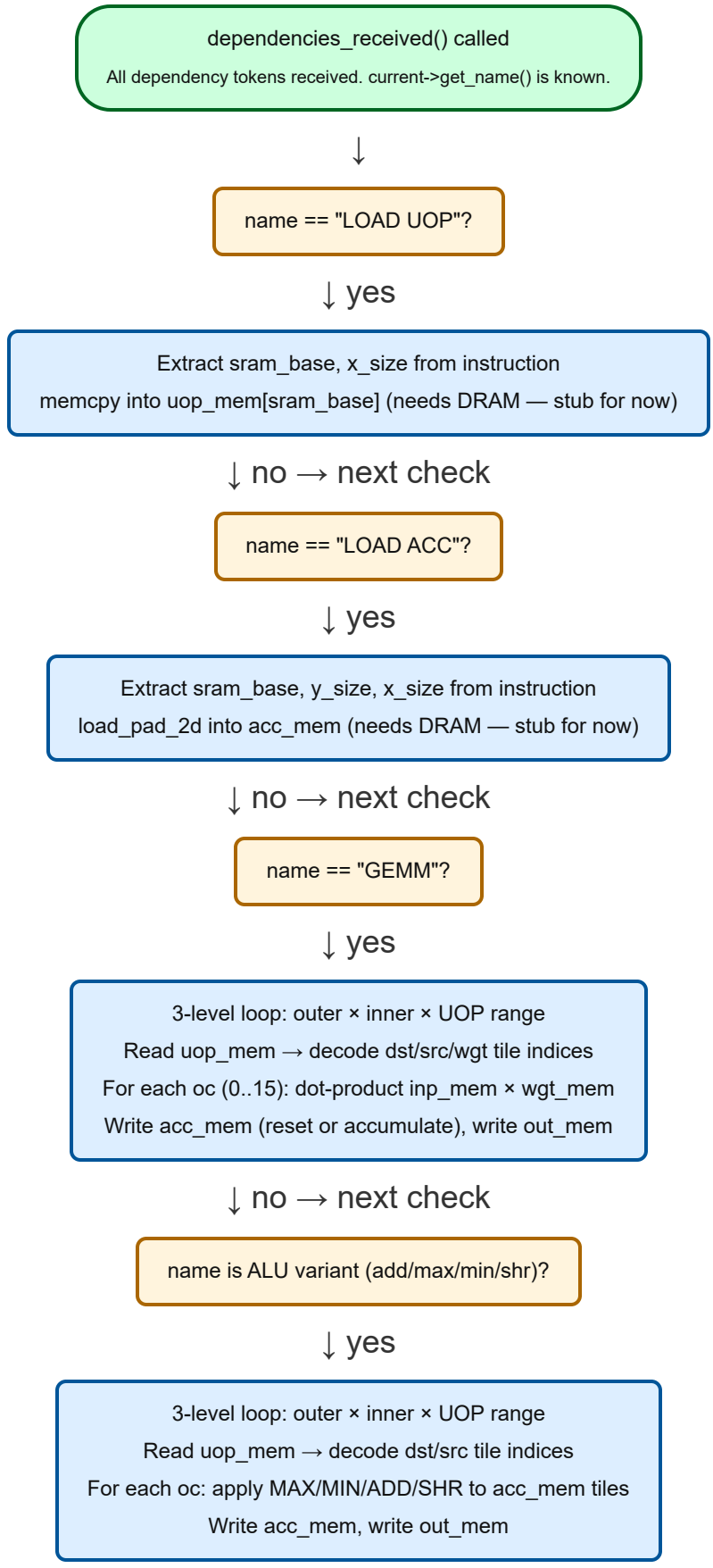
```
// ── ALU (add / add imm / max imm / min imm / shr) ──────────────────────────
// Mirrors: EXE_OUT_LOOP > EXE_IN_LOOP > READ_ALU_UOP > opcode switch from vta.cc alu()
else if (name != "NOP-COMPUTE-STAGE" && name != "FINISH") {

    // Detect use_imm from instruction name (Instruction class has no get_imm() getter)
    bool use_imm = (name.find("imm") != std::string::npos);
    int32_t imm  = 0;  // assumed 0 (covers ALU-max imm = ReLU: max(x, 0))

    int dst_offset_out = 0;   // vta.cc: acc_idx_T dst_offset_out = 0
    int src_offset_out = 0;   // vta.cc: inp_idx_T src_offset_out = 0

    // EXE_OUT_LOOP
    for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
        int dst_offset_in = dst_offset_out;
        int src_offset_in = src_offset_out;

        // EXE_IN_LOOP
        for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

            // READ_ALU_UOP
            for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                uint32_t uop = uop_mem[upc];

                // ALU UOP: [10:0]=dst [21:11]=src  (no wgt field)
                int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
                int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;

                for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {
                    // vta.cc: acc_T src_0 = dst_tensor[i][b]
                    int32_t src_0 = acc_mem[dst_idx][oc];
                    // vta.cc: acc_T src_1 = insn.use_imm ? (acc_T)insn.imm : src_tensor[i][b]
                    int32_t src_1 = use_imm ? imm : acc_mem[src_idx][oc];

                    int32_t result;
                    // vta.cc: if (alu_opcode == MIN || MAX) ... else if ADD ... else if SHR
                    if      (name.find("max") != std::string::npos) result = std::max(src_0, src_1);
                    else if (name.find("min") != std::string::npos) result = std::min(src_0, src_1);
                    else if (name.find("add") != std::string::npos) result = src_0 + src_1;
                    else                                            result = src_0 >> src_1; // shr

                    // vta.cc: dst_tensor[i][b] = result; o_tensor[i][b] = result.range(7,0)
                    acc_mem[dst_idx][oc] = result;
                    out_mem[dst_idx][oc] = (int8_t)(result & 0xFF);
                }
            }

            // vta.cc: dst_offset_in += insn.dst_factor_in; src_offset_in += insn.src_factor_in
            dst_offset_in += current->get_alu_inner_loop_dst();
            src_offset_in += current->get_alu_inner_loop_src();
        }

        // vta.cc: dst_offset_out += insn.dst_factor_out; src_offset_out += insn.src_factor_out
        dst_offset_out += current->get_alu_outer_loop_dst();
        src_offset_out += current->get_alu_outer_loop_src();
    }
}
// NOP-COMPUTE-STAGE and FINISH: no computation, timing only
```

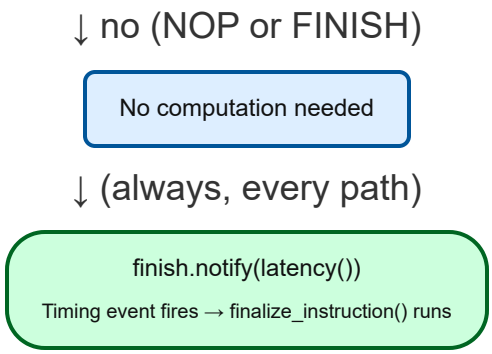
8. Complete dependencies_received() — Final Code

```
void ComputeModule::dependencies_received() {
    std::string name = current->get_name();

    // ── LOAD UOP: fill uop_mem from DRAM ──────────────────────────────
    if (name == "LOAD UOP") {
        // vta.cc: memcpy(&uop_mem[sram_base], &uops[dram_base], x_size * sizeof(uop_T))
        // Simulator has no DRAM pointer — uop_mem stays 0. UOP data needed for full sim.
        int sram_base = (int)std::stoul(current->get_sram(), nullptr, 16);
        (void)sram_base;
    }

    // ── LOAD ACC: pre-load bias into acc_mem from DRAM ────────────────
    else if (name == "LOAD ACC") {
        // vta.cc: load_pad_2d(biases, acc_mem, sram_base, dram_base, y_size, x_size, ...)
        // Simulator has no DRAM pointer — acc_mem stays 0 (zero bias).
        int sram_base = (int)std::stoul(current->get_sram(), nullptr, 16);
        (void)sram_base;
    }

    // ── GEMM: inp × wgt → acc, then acc → out ─────────────────────────
    else if (name == "GEMM") {
        int dst_offset_out = 0, src_offset_out = 0, wgt_offset_out = 0;

        for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
            int dst_offset_in = dst_offset_out;
            int src_offset_in = src_offset_out;
            int wgt_offset_in = wgt_offset_out;

            for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {
                for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                    uint32_t uop = uop_mem[upc];
                    int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
                    int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;
                    int wgt_idx = (int)((uop >> (vta_config::UOP_DST_WIDTH + vta_config::UOP_SRC_WIDTH)) & WG

                    for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {
                        int32_t accum = acc_mem[dst_idx][oc];
                        int32_t tmp   = 0;
                        for (int ic = 0; ic < VTA_BLOCK_IN; ic++) {
                            tmp += (int32_t)inp_mem[src_idx][ic] * (int32_t)wgt_mem[wgt_idx][oc * VTA_BLOCK_I
                        }
                        accum += tmp;
                        acc_mem[dst_idx][oc] = current->get_reset_out() ? 0 : accum;
                        out_mem[dst_idx][oc] = (int8_t)(accum & 0xFF);
                    }
                }
                dst_offset_in += current->get_gemm_inner_loop_acc();
                src_offset_in += current->get_gemm_inner_loop_inp();
                wgt_offset_in += current->get_gemm_inner_loop_wgt();
            }
            dst_offset_out += current->get_gemm_outer_loop_acc();
            src_offset_out += current->get_gemm_outer_loop_inp();
            wgt_offset_out += current->get_gemm_outer_loop_wgt();
        }
```

```
    }

    // ── ALU (all variants: add / add imm / max imm / min imm / shr) ──────────
    else if (name != "NOP-COMPUTE-STAGE" && name != "FINISH") {
        bool use_imm = (name.find("imm") != std::string::npos);
        int32_t imm  = 0;

        int dst_offset_out = 0, src_offset_out = 0;

        for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
            int dst_offset_in = dst_offset_out;
            int src_offset_in = src_offset_out;

            for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {
                for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                    uint32_t uop = uop_mem[upc];
                    int dst_idx = (int)((uop >> 0) & DST_MASK) + dst_offset_in;
                    int src_idx = (int)((uop >> vta_config::UOP_DST_WIDTH) & SRC_MASK) + src_offset_in;

                    for (int oc = 0; oc < VTA_BLOCK_OUT; oc++) {
                        int32_t src_0  = acc_mem[dst_idx][oc];
                        int32_t src_1  = use_imm ? imm : acc_mem[src_idx][oc];
                        int32_t result;

                        if      (name.find("max") != std::string::npos) result = std::max(src_0, src_1);
                        else if (name.find("min") != std::string::npos) result = std::min(src_0, src_1);
                        else if (name.find("add") != std::string::npos) result = src_0 + src_1;
                        else                                            result = src_0 >> src_1;

                        acc_mem[dst_idx][oc] = result;
                        out_mem[dst_idx][oc] = (int8_t)(result & 0xFF);
                    }
                }
                dst_offset_in += current->get_alu_inner_loop_dst();
                src_offset_in += current->get_alu_inner_loop_src();
            }
            dst_offset_out += current->get_alu_outer_loop_dst();
            src_offset_out += current->get_alu_outer_loop_src();
        }
    }
    // NOP-COMPUTE-STAGE / FINISH: no computation here — handled by finalize_instruction()

    finish.notify(latency());  // always at the end — fires timing event
}
```

9. Complete Dataflow Diagram




↓ no (NOP or FINISH)
No computation needed
↓ (always, every path)
finish.notify(latency())
Timing event fires → finalize_instruction() runs

10. Summary: What Goes Where

File	What you add	Notes
compute_module.cpp	Add `#include <cstring>` and `#include <algorithm>`	Needed for memcpy, std::max, std::min
compute_module.cpp	6 `constexpr` constants (VTA_BATCH, VTA_BLOCK_IN, VTA_BLOCK_OUT, buffer depths)	After includes, before any function
compute_module.cpp	3 mask constants (DST_MASK, SRC_MASK, WGT_MASK) using vta_config:: bit widths	Replaces 0x7FF / 0x3FF raw numbers
compute_module.cpp	`static uint32_t uop_mem[...]` and `static int32_t acc_mem[...]`	Private to Compute, mirrors vta.cc static declarations
compute_module.cpp	`int8_t inp_mem[...]`, `wgt_mem[...]`, `out_mem[...]` (non-static)	Global symbols — Load and Store modules can access with `extern`
compute_module.cpp	Full `dependencies_received()` body (Section 8)	Replaces the single `finish.notify(latency())` line
load_module.cpp	`extern int8_t inp_mem[][16];` `extern int8_t wgt_mem[][256];`	So LoadModule can fill them. No .h change needed.
store_module.cpp	`extern int8_t out_mem[][16];`	So StoreModule can read it. No .h change needed.
Any .h file	*Nothing* — professor confirmed all declarations are already there	—